Diseño y Análisis de Algoritmos

Ayudantía 5: Programación Dinámica

Ayud. Ferran Rojas Andreu

1. Idea general

Sea $F(i)$ el mínimo número de monedas necesario para pagar exactamente el monto i, usando solo monedas de $D = \{1, 3, 4\}$.

La observación clave de la programación dinámica es que, para pagar i, la *última* moneda usada debe ser alguna denominación $c \in D$ con $c \leq i$. Si esa moneda es c, entonces antes de usarla debíamos haber pagado $i - c$ con $F(i - c)$ monedas. Por lo tanto:

$$F(i) \;=\; 1 + \min_{\substack{c \in D \\ c \leq i}} F(i - c), \qquad F(0) = 0.$$

2. (a) Cálculo a mano para $A = 6$

Caso base:

$$F(0) = 0.$$

$$\begin{aligned}
&F(1) = 1 + F(0) = 1 && \text{(moneda 1)}\\
&F(2) = 1 + F(1) = 2 && \text{(solo } 1+1\text{)}\\
&F(3) = \min\{1 + F(2),\ 1 + F(0)\} = \min\{3, 1\} = 1 && \text{(moneda 3)}\\
&F(4) = \min\{1 + F(3),\ 1 + F(1),\ 1 + F(0)\} = \min\{2, 2, 1\} = 1 && \text{(moneda 4)}\\
&F(5) = \min\{1 + F(4),\ 1 + F(2),\ 1 + F(1)\} = \min\{2, 3, 2\} = 2 && (4+1)\\
&F(6) = \min\{1 + F(5),\ 1 + F(3),\ 1 + F(2)\} = \min\{3, 2, 3\} = 2 && (3+3)
\end{aligned}$$

i	0	1	2	3	4	5	6
$F(i)$	0	1	2	1	1	2	2

Valor final:

$$F(6) = \boxed{2} \qquad \text{(por ejemplo, } 3+3 \text{ o } 4+1+1\text{)}.$$

3. (b) Definición de la recurrencia

$$F(i) = \begin{cases} 0, & \text{si } i = 0, \\ 1 + \min\limits_{\substack{c \in D \\ c \leq i}} F(i - c), & \text{si } i \geq 1, \end{cases} \qquad D = \{1, 3, 4\},\ i \in \{0, 1, \ldots, A\}.$$

4. (c) Implementación bottom-up

MinCoins($D[1..k]$, A)

```
1  Crear arreglo F[0..A]
2  F[0] ← 0
3  for i = 1 to A
4      F[i] ← ∞
5      for j = 1 to k
6          if D[j] ≤ i
7              F[i] ← mín (F[i], 1 + F[i − D[j]])
8  return F[A]
```

Complejidad. El bucle exterior recorre $A+1$ montos y, por cada uno, el bucle interior prueba las k denominaciones; cada operación es $O(1)$. Luego el tiempo es $O(A \cdot k)$. El arreglo F de tamaño $A+1$ usa $O(A)$ espacio adicional.

Correctitud (esquema). Por inducción sobre i: para $i = 0$ es trivial. Si $F(j)$ es óptimo para todo $j < i$, entonces toda solución óptima para i termina en alguna moneda c, de modo que su costo es $1 + F(i-c)$; el mínimo sobre c cubre todas las opciones, y como $F(i-c)$ es óptimo por hipótesis, $F(i)$ es óptimo. ■

5. (a) Cálculo a mano de $F(i)$ para $A = \langle 10, 9, 2, 5, 3, 7, 101, 18 \rangle$

Sea $F(i)$ la longitud de la LIS que *termina* en $A[i]$. Toda subsecuencia que termina en $A[i]$ está formada por una subsecuencia que termina en algún $A[j]$ con $j < i$ y $A[j] < A[i]$, más el propio $A[i]$. Por lo tanto:

$$F(i) = 1 + \max_{\substack{j<i \\ A[j]<A[i]}} F(j), \qquad F(i) = 1 \text{ si no existe tal } j.$$

i	1	2	3	4	5	6	7	8
$A[i]$	10	9	2	5	3	7	101	18
$F(i)$	1	1	1	2	2	3	4	4

Detalle de los pasos no triviales:

$F(4) = 1 + F(3) = 2$	pues $2 < 5$
$F(5) = 1 + F(3) = 2$	pues $2 < 3$
$F(6) = 1 + \max\{F(4), F(5)\} = 1 + 2 = 3$	pues $5 < 7$ y $3 < 7$
$F(7) = 1 + F(6) = 4$	pues $7 < 101$
$F(8) = 1 + F(6) = 4$	pues $7 < 18$

Valor final (longitud de la LIS):

$$\boxed{\max_i F(i) = F(7) = 4}.$$

6. (b) Recurrencia

$$F(i) = \begin{cases} 1, & \text{si } \nexists j < i \text{ con } A[j] < A[i], \\ 1 + \max\limits_{\substack{j<i \\ A[j]<A[i]}} F(j), & \text{si } \exists j < i \text{ con } A[j] < A[i], \end{cases} \qquad i \in \{1, \ldots, n\}.$$

La respuesta al problema es $\max_{1 \le i \le n} F(i)$.

7. (c) Implementación bottom-up en $O(n^2)$

```
LIS(A[1..n], n)
1  Crear arreglo F[1..n]
2  mejor ← 1
3  for i = 1 to n
4      F[i] ← 1
5      for j = 1 to i − 1
6          if A[j] < A[i]
7              F[i] ← máx (F[i], F[j] + 1)
8      mejor ← máx(mejor, F[i])
9  return mejor
```

Complejidad. El bucle exterior recorre n posiciones y el bucle interior hasta $i-1$ posiciones; el total es $\sum_{i=1}^{n}(i-1) = \frac{n(n-1)}{2} = O(n^2)$ operaciones. El arreglo F usa $O(n)$ espacio adicional.

Reconstrucción (opcional). Guardando un arreglo π donde $\pi[i]$ es el índice $j < i$ que maximiza $F(i)$ (o NIL), se reconstruye una LIS siguiendo los predecesores desde el índice del máximo. Para el ejemplo, la LIS $2, 3, 7, 101$ (índices $3, 5, 6, 7$) es una de las posibles.

8. (a) Tabla $F(i,j)$ para $w=(2,3,4,5)$, $v=(3,4,5,6)$, $W=8$

Sea $F(i,j)$ el máximo valor usando los primeros i objetos con capacidad j. Caso base: $F(0,j)=0$ para todo j (sin objetos no hay valor) y $F(i,0)=0$ para todo i (sin capacidad no cabe nada).

$i\backslash j$	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1 (w=2, v=3)	0	0	3	3	3	3	3	3	3
2 (w=3, v=4)	0	0	3	4	4	7	7	7	7
3 (w=4, v=5)	0	0	3	4	5	7	8	9	9
4 (w=5, v=6)	0	0	3	4	5	7	8	9	**10**

Valor final:

$$\boxed{F(4,8)=10}.$$

Reconstrucción del subconjunto óptimo: $F(4,8)=F(3,3)+6$ (se toma el objeto 4), $F(3,3)=F(2,3)=4$ (no cabe el 3), $F(2,3)=F(1,0)+4$ (se toma el objeto 2), $F(1,0)=0$ (el 1 no cabe). Luego $S=\{2,4\}$, con peso $3+5=8\le W$ y valor $4+6=10$. ✓

9. (b) Recurrencia

Para $1\le i\le n$ y $0\le j\le W$:

$$F(i,j)=\begin{cases} F(i-1,j), & \text{si } w_i>j \text{ (el objeto } i \text{ no cabe)},\\ \max\big(F(i-1,j),\ F(i-1,j-w_i)+v_i\big), & \text{si } w_i\le j, \end{cases} \qquad F(0,j)=0, \quad F(i,0)$$

Interpretación. Cuando el objeto i cabe, hay dos opciones: *no tomarlo* (heredar el mejor valor sin el objeto, $F(i-1,j)$) o *tomarlo* (sumar v_i al mejor valor con capacidad restante $j-w_i$, $F(i-1,j-w_i)+v_i$); se elige el máximo. Cuando no cabe, solo queda la primera opción. La respuesta es $F(n,W)$.

10. (c) Implementación bottom-up en $O(n\,W)$

```
KNAPSACK01(w[1..n], v[1..n], n, W)
1  Crear tabla F[0..n][0..W] inicializada en 0
2  for i = 1 to n
3      for j = 1 to W
4          if w[i] > j
5              F[i][j] ← F[i − 1][j]
6          else
7              F[i][j] ← max (F[i − 1][j], F[i − 1][j − w[i]] + v[i])
8  return F[n][W]
```

Complejidad. Hay $n\cdot W$ celdas y cada una se calcula en $O(1)$; el tiempo es $O(n\,W)$. La tabla F usa $O(n\,W)$ espacio adicional. *(Optimización:* como $F(i,\cdot)$ solo depende de la fila anterior, basta un arreglo de $W+1$ entradas recorrido de derecha a izquierda, reduciendo el espacio a $O(W)$.)

Correctitud (esquema). Inducción sobre i. Para $i = 0$ es trivial. Para el objeto i, toda solución óptima con capacidad j lo incluye o no. Si no lo incluye, su valor es $F(i-1, j)$. Si lo incluye, su valor es v_i más una solución óptima con los $i-1$ objetos anteriores y capacidad $j - w_i$, a saber $F(i-1, j-w_i)$. Tomar el máximo de ambas cubre todas las posibilidades y, por hipótesis inductiva, ambos términos son óptimos. ■